EC 504
Fall, 2025
Homework 3

Due Thursday, Oct. 16, 2025, 11:59 pm, on your SCC account

Problem 3.1 (20 pts) Quick questions: Answer whether each of the statements below is true or false, and give a quick explanation.

(a) If the range of a list of numbers is bounded by a constant M, there are algorithms that can sort the list in $O(\log(n))$, where n is the length of the list.

Solution:

True. Radix sort, for instance, would do it.

(b) Assume we create a hybrid algorithm using mergesort and insertion sort, where we use insertion sort to sort lists that are 32 elements or less long, and we use mergesort to merge the sorted lists hierarchically. The asymptotic complexity of this hybrid algorithm is $O(n \log n)$.

Solution:

True. Insertion sort will take $32n$ operations to sort the initial sublists. Mergesort will take $\log(n)$ merge operations, each of which will be $O(n)$. Hence, the total worst case complexity is $O(n \log(n))$.

(c) If the height of a binary tree is the maximum number of edges in any root to a leaf path, then the maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.

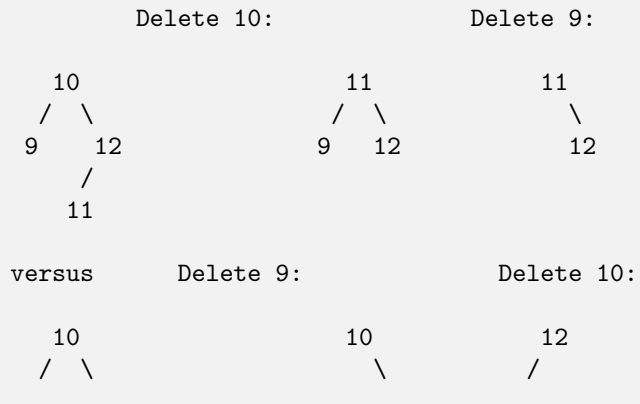
Solution:

True. The maximum number would be in a perfect binary tree with all levels complete. A perfect binary tree of height h has $2^{h+1} - 1$ nodes.

(d) In a binary search tree with no repeated keys, deleting the node with key x, followed by deleting the node with key y, will result in the same search tree as deleting the node with key y, then deleting the node with key x.

Solution:

The answer to this is false, because of how we delete nodes with only one child. A simple counter-example is



```
    9    12                12       11
        /                 /
       11                11
```

Note that delete 9, then 10 results in a different tree than delete 10, then 9.

(e) Suppose you have an unsorted array of n elements, where $n = m^2$ for some integer m. One can determine the m smallest elements of the array in $O(n)$ time.

Solution:

This is true; there are several algorithms to do this, but you use the algorithm for finding the k-th smallest element in $O(n)$ using the median of medians techniques. Once you find that element, do a pass through the list again and find all elements smaller than this. Two passes, each $O(n)$.

(f) The second smallest element in a min-heap with distinct values must be a child of the root.

Solution:

That is true. If it were not, then the heap property is violated.

(g) In a min-heap with distinct values, the largest element is always on a leaf.

Solution:

Yes, because otherwise you violate the min-heap property.

(h) Inserting numbers $1, \ldots, n$ into a binary min-heap in that order will take $O(n)$ time.

Solution:

Yes, because you never have to upheap the insertions, as the elements are already in order.

(i) Is the array $(1, 2, 6, 4, 7, 9, 10, 3, 5)$ a min-heap?

Solution:

No, heap property is violated by children of 4.

(j) Is the binary tree generated by a Huffman code always a full binary tree?

Solution:

Yes, because the last bit in each code must separate two symbols.

Problem 3.2 (20 pts)

(a) Given the following list of elements,

$$35, 22, 11, 98, 55, 66, 77, 80, 85, 90, 1$$

Draw the sequence of binary min-heaps (or write the equivalent arrays) which results from inserting the following values in the order in which they appear into an empty heap.

Solution:

We will write the arrays. The trees follow directly:'

$$[35] \to [22, 35] \to [11, 35, 22] \to [11, 35, 22, 98] \to [11, 35, 22, 98, 55] \to [11, 35, 22, 98, 55, 66]$$
$$\to [11, 35, 22, 98, 55, 66, 77] \to [11, 35, 22, 80, 55, 66, 77, 98] \to [11, 35, 22, 80, 55, 66, 77, 98, 85]$$
$$\to [11, 35, 22, 80, 55, 66, 77, 98, 85, 90] \to [1, 11, 22, 80, 35, 66, 77, 98, 85, 90, 55]$$

(b) Remove the top of the heap (smallest element), and show the steps to get the heap that results after this removal.

Solution:

We will write the arrays. The trees follow directly:'

$$[1, 11, 22, 80, 35, 66, 77, 98, 85, 90, 55] \to [55, 11, 22, 80, 35, 66, 77, 98, 85, 90]$$
$$\to [11, 55, 22, 80, 35, 66, 77, 98, 85, 90] \to [11, 35, 22, 80, 55, 66, 77, 98, 85, 90]$$

(c) Remove the top element of the remaining heap and show the heap that results.

Solution:

We will write the arrays. The trees follow directly:'

$$[11, 35, 22, 80, 55, 66, 77, 98, 85, 90] \to [90, 35, 22, 80, 55, 66, 77, 98, 85]$$
$$\to [22, 35, 90, 80, 55, 66, 77, 98, 85] \to [22, 35, 66, 80, 55, 90, 77, 98, 85]$$

Problem 3.3 (20 pts)

Here is a classical problem that is often part of interviews. You are given n nuts and n bolts, such that one and only one nut fits each bolt. Your only means of comparing these nuts and bolts is with a function TEST(x, y), where x is a nut and y is a bolt. The function returns $+1$ if the nut is too big, 0 if the nut fits, and -1 if the nut is too small. Note that you cannot compare nuts to nuts and bolts to bolts.

The usual question is to design and analyze an algorithm for sorting the nuts and bolts from smallest to largest using the TEST function. While there are many approaches that work, the smart answer is to combine a pivot operation from quicksort to solve this as follows: Pick a bolt, and partition the nut array into nuts that are smaller, nut that fits, and nuts that are larger. Then, pick the matching nut, and partition the nut array similarly. Below is pseudo-code for this algorithm.

```
Procedure quicknutboltsort(nutarray,boltarray,m,n):
if m >=n, return;
new_nutarray[0:n-m] = -1; new_boltarray[0:n-m]=-1;
testbolt = boltarray[n];
low = 0; best_fit = -1; high = n-m;
For k in m to n,
     if Test(nutarray[k],testbolt) < 0,  // nut too small, move to front
          new_nutarray[low++]=nutarray[k];
     else if Test(nutarray[k],testbolt) == 0, //nut fits, remember it
          best_fit = nutarray[k];
```

```
     else  //nut is too big, move to back
          new_nutarray[high--] = nutarray[k];
new_nutarray[low] = best_fit;
// Now pivot the bolts against the best_fit nut.
low = 0; testnut = best_fit; best_fit = -1; high = n-m;
For j in m to n,
     if Test(testnut,boltarray[j]) < 0, //bolt is too big, move to back
          new_boltarray[high--]=boltarray[j];
     else if Test(testnut,boltarray[j]) > 0, // bolt is too small, move to front
          new_boltarray[low++]=boltarray[j];
     else
         continue;
new_boltarray[low] = testbolt;
// now recur:
quicknutboltsort(new_nutarray, new_boltarray, 0, low-1);
quicknutboltsort(new_nutarray,new_boltarray,low+1,n-m);
for k in m to n:
    nutarray[k] = newnut_array[k-m];
    boltarray[k] = newbolt_array[k-m];
```

(a) What is the tightest worst-case asymptotic growth of the above function as $n \to \infty$?

Solution:

The worst case happens when nuts are sorted in increasing order, and the bolts are sorted in decreasing order. In that case, you go through the entire list at each pass, and the recursion is

$$T(n) = T(n-1) + T(0) + cn$$

This means $T(n) = c(1 + c + 2c + 3c + \ldots + nc \in O(n^2)$.

(b) The above algorithm uses lots of extra storage, copying arrays back and forth. Modify the above algorithm so that you use only the original nutarray and boltarray.

Solution:

See below:

```
Procedure quicknutboltsort(nutarray,boltarray,m,n):
if m >=n, return;
testbolt = boltarray[n];
low = m;
//Pivot the nuts against the last bolt, moving smaller nuts to front
For k in m to n-1,  //leave the last position for the nut that fits
     if Test(nutarray[k],testbolt) < 0, // nut too small, move to front
         swap(nutarray[k],nutarray[low])
           low++;
     else if Test(nut_array[k],testbolt) == 0, //nut fits, swap with last and retest k
           swap(nutarray[k],nutarray[n])
           k--;
swap(nutarray[low], nutarray[n]); // swap last nut into correct position

// Now pivot the bolts against the nut that fit testbolt
testnut = nutarray[low];
low = m;
```

```
For j in m to n-1, //we know the bolt in last position is the one that fits
     if Test(testnut,boltarray[j]) > 0, // bolt too small, swap down
          swap(boltarray[j],boltarray[low]);
          low++;
     else if Test(testnut,boltarray[j]) == 0, // not needed, left it in for symmetry
          swap(boltarra[j], boltarray[n])
          j--;
swap(boltarray[low], boltarray[n]) // put bolt that fits in correct position
// now recur, excluding the nut-bolt position that already fits.
quicknutboltsort(nutarray, boltarray, m, low-1);
quicknutboltsort(nutarray,boltarray,low+1,n);
```

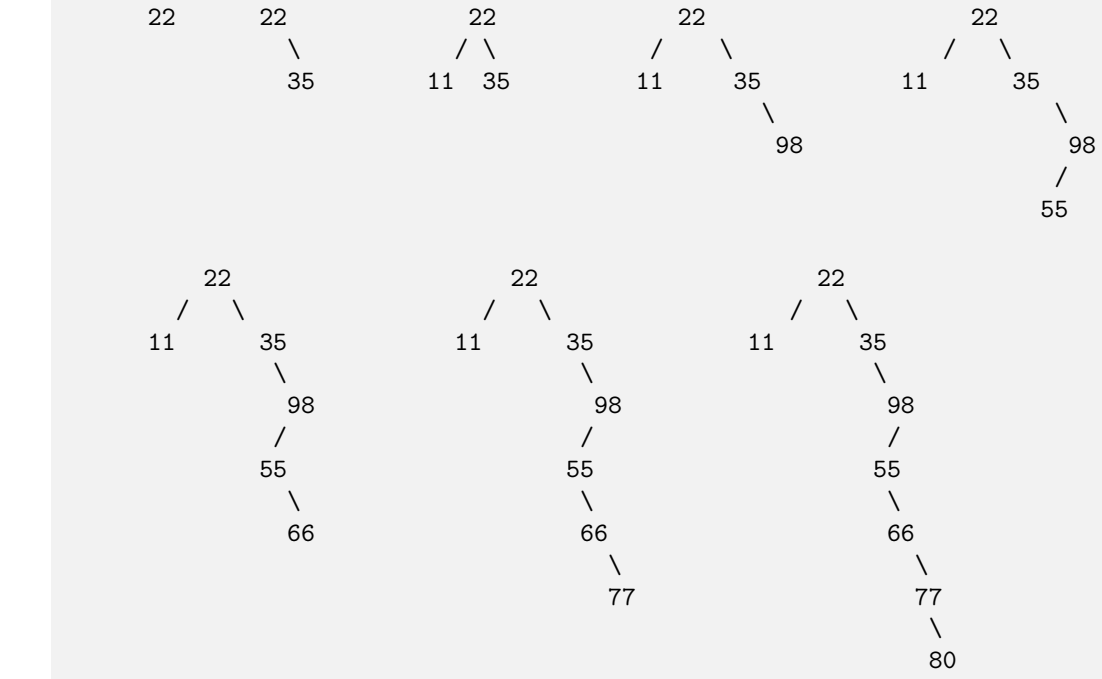
Problem 3.4 (10 pts)

Consider the following list of elements:

$$22, 35, 11, 98, 55, 66, 77, 80$$

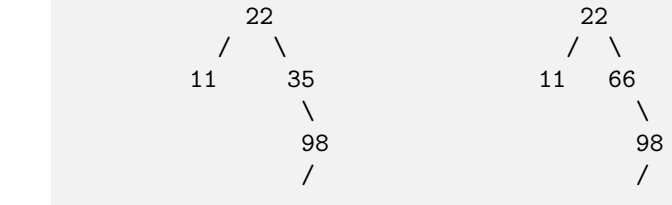
(a) Insert the following items into a binary search tree, in the order given. Show the binary trees after each insert.

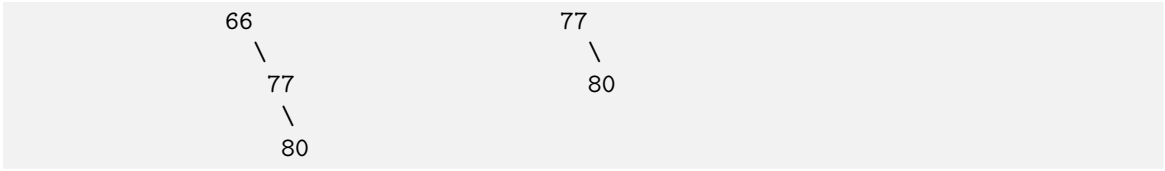
Solution:



(b) From the last tree in the previous part, show the tree that remains when you delete 55. Then show the tree that results when you delete 35.

Solution:





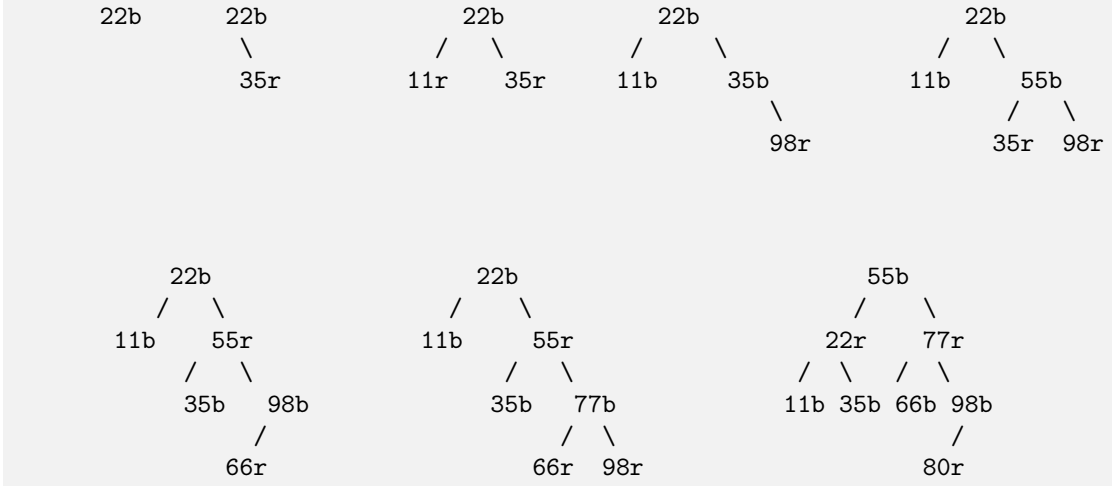
Problem 3.5 (10 pts)

Consider the following list of elements:

$$22, 35, 11, 98, 55, 66, 77, 80$$

1. Insert the following items into a red-black tree, in the order given. Show the red-black trees after each insert. Put a b next to each black node, r next to each red node (or use color pens...)

Solution:



2. From the last tree above, show the tree that remains when you delete 55.

Solution:

Problem 3.6 (20 pts)

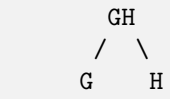
You are interested in compression. Given a file with characters, you want to find the binary code which satisfies the prefix property (no conflicts) and which minimizes the number of bits required. As an example, consider an alphabet with 8 symbols, with relative weights (frequency) of appearance in an average text file give below:

Symbols:	A	L	G	O	R	I	T	H
Weights:	68	20	5	30	18	15	19	12

(a) Determine the Huffman code by constructing a tree that gives the minimum number of characters to code the letters. Arrange tree with smaller weights to the left.

Solution:

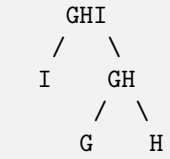
First we group the two least frequent keys: G and H, under a node, and add an aggregate key GH with total occurrence 17. The partial tree is



Our new problem is now

Symbols:	A	L	GH	O	R	I	T
Weights:	68	20	17	30	18	15	19

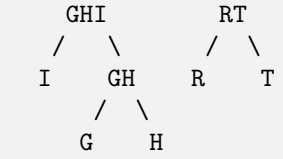
We group GH and I as the two smallest. The new partial tree is



We add new symbol GHI with 32 occurrences. Our new problem is now

Symbols:	A	L	GHI	O	R	T
Weights:	68	20	32	30	18	19

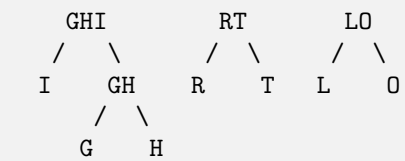
Group R and T. Our partial solution is:



Aggregate, resulting in new problem

Symbols:	A	L	GHI	O	RT
Weights:	68	20	32	30	37

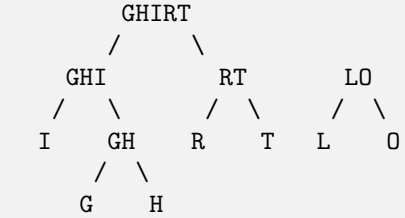
Merge L and O:



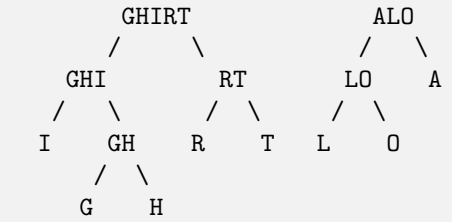
New problem

Symbols:	A	LO	GHI	RT
Weights:	68	52	32	37

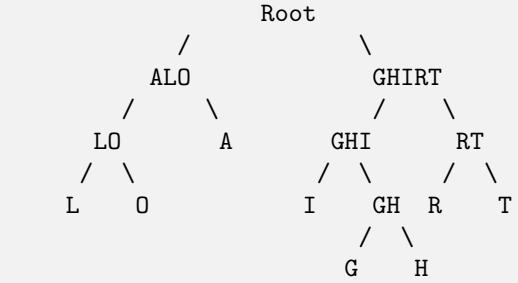
Merge GHI and RT:



New problem	Symbols:	A	LO	GHIRT	Merge A and LO:
	Weights:	68	52	69	



then merge everything into one tree:



(b) Identify the code for each letter and list the number of bits for each letter and compute the average number of bits per symbols in this code. Is it less than 3? (You can leave the answer as a fraction since getting the decimal value is difficult without a calculator.)

Solution:

Let's assume for convention that going left is 0, right is 1. The resulting code is:

```
A:  11  (2 bits)
I:  00  (2 bits)
G: 0010 (4 bits)
H: 0011 (4 bits)
R: 010  (3 bits)
T: 011  (3 bits)
L: 100  (3 bits)
O: 101  (3 bits)
```

Total number of bits: 495, total number of symbols: 187, so average bits per symbol is 2.647.

(c) Give an example of weights for these 8 symbols that would saturate 3 bits per letter. What would the Huffman tree look like?

Solution:

The simplest example is when all weights are the same. In this case, we get a perfect binary tree of height 3.